

Finance Radar 组内工作说明

我们在做什么，为什么需要你，以及你具体要交出什么

2026-08-16 · 发给全体组员 · 数据核对至代码提交 ceb9f57 / 生产版本 2026.08.15.4

0. 先说清楚这份文件是干什么的

我们组的选题是一个已经在公网上跑着的真实系统，不是从零开始写的课程作业。这带来一个好处和一个麻烦。

好处是：我们的起点比大多数组高很多。系统已经上线、有真实数据、有完整测试和部署，直接按"竞赛/高难度创新级"（评分上限 100 分）去申报是站得住的。

麻烦是：目前工程主体是一个人做出来的。而这门课的评分规则里，个人成绩要乘一个 0.8–1.2 的贡献系数，由组内互评和导师评估决定，制度目的写得很直白——“避免划水与带飞同分”。也就是说：项目做得好不会自动变成你的分数。你必须有自己的代码、自己的测试、自己能讲的东西。

所以这份文件要回答三件事：

1. 我们做的到底是什么东西，为什么它值得按 100 分档去申报；
2. 已经有什么了（你不需要重做），还缺什么（只能由你们来补）；
3. 你个人具体要交出哪些东西，才能拿到该拿的分。

技术细节（系统怎么运转、数据库长什么样、AI 用在哪）另有一份《Finance Radar 组内说明书》，17 页，建议开工前通读一遍。这份文件只讲“我们在做什么”和“你做什么”。

1. 我们在做什么

一句话

我们做的是个金融事件的“证据雷达”：它每 5 分钟自动去 13 个官方网站（美联储、SEC、CFTC、FDA、FTC、FDIC、欧洲央行、美国能源署、美国劳工统计局）和 1 个聚合新闻源抓消息，把消息变成“事件”，给每个事件配上能点回原文的精确引文，记录这个事件后来被官方修订、更正、撤回的全过程，并且在证据不够或者互相矛盾的时候，明确地说“我不下结论”。

它不是炒股软件。代码里没有下单、持仓、余额这些东西，而且这一点是被数据库约束和 CI 强制保证的，不靠自觉。

它解决的问题

金融新闻传播极快，但普通人几乎没法验证。一条“某公司要破产了”的消息，可能来自：官方文件、公司自己的公告、财经媒体转述、社交平台传闻。这四种东西的可信度差着量级，但在你的信息流里长得一模一样。更

麻烦的是，官方文件会被更正、会被撤回，而转述它的新闻不会跟着撤回。

我们做的事情是：把"消息如何变成结论"这个过程完整保存下来，让人能一路点回原始证据；并且在证据不足时拒绝下结论，而不是给一个看起来很确定的答案。

记住这句话，答辩会反复用到

别的项目做的是"让 AI 猜涨跌"，我们做的是"把消息怎么变成结论完整保存下来，并且在证据不足或冲突时拒绝下结论"。

为什么这题能按 100 分档申报

课程规则里对项目分级有明确标准，我们逐条对一下：

规则要求	我们的情况
AI 深度融入核心业务，禁止"主营业务 + 附加聊天框"的浅层集成	AI 参与的是事件候选排序和证据摘要，都在主流程里。而且最终结论由确定性证据门和人工裁决决定，不是模型说了算——这比"挂个聊天框"深得多，也更难做
淘汰低价值 CRUD	整个系统没有一个"增删改查"页面。核心是事件版本链、证据图、修订追踪和门禁状态机
创新性、技术深度高	证据门 + 修订追踪 + 诚实弃权 + 模型失败治理 + 独立产品 KPI，五个点都不是常规课程项目会做的
成果可直接用于竞赛或高质量展示	公网可访问、可断网离线演示、有完整审计链
可自拟创新项目或带已有半成品深化（规则原文）	我们正是这一条。规则明确允许，Day 1 下午当面向教师说明来源即可

兜底方案：如果教师不认可高难度定级，退到课程标准题库的 A10 多源新闻聚合 Agent（上限 95 分），我们现有的东西完全覆盖它的要求。

2. 现在到哪一步了

结论先给：工程侧已经在优秀档，课程侧一片空白。我们丢分的地方不在技术。

2.1 已经有的（你不需要重做）

板块	状态	具体是什么
生产部署	在线	AWS 上 systemd 分离运行 API / 公开 Web / 采集 Worker / 每日备份，公网 HTTPS 可访问
数据规模	持续增长	13,357 条事件，其中 7,471 条待核验；Worker 每 5 分钟一轮
代码规模	—	202 个 Python 文件，约 66,700 行（应用 13,972 / 脚本 34,153 / 测试 18,575）
测试	全绿	669 项通过 + 20 项子测试，0 失败（写这份文件时实跑的，30 秒跑完）
接口	完成	25 条 API 路由，三层令牌权限，每个响应带版本号 + 追踪 ID + 生成时间
界面	完成	6 个页面 × 4 种身份隔离（公开 / 复核 / 运维 / 管理员）
安全边界	0 违规	6 条数据库 CHECK 约束把“不碰交易”写死在存储层
备份恢复	已验证	每日备份 + 隔离环境完整恢复验证 + SHA-256 清单
断网演示	已冻结	4 个冻结回放案例 + 可执行离线终端
故障演练	6/6	畸形 XML、模型损坏、备份损坏、证据门、官方更正撤回、禁交易路由
模型治理	已修复并自限	见下方 2.2，这是我们最强的一张牌
产品 KPI	已实现	见下方 2.3，很多人会忽略但很值钱

2.2 模型治理：我们最强的一张牌

系统里有一个“风险路由”小模型，负责把可能有风险的事件排到复核队列前面。它经历过一次彻底的失败和一次彻底的重做，全过程都留了档。

版本	盲测集	准确率	风险召回	普通新闻误报率	门禁
v1	40 条	50.0%	100%	95.0%	FAIL
v3	80 条	56.2%	40.0%	6.7%	FAIL
v4（当前生产）	80 条	96.3%	96.7%	6.7%	PASS 11/11

每一代的盲测集都是先冻结标准答案、再跑模型，并且和训练数据零重叠——这样才不会自己给自己放水。

v1 错在哪：它对 40 条里的 39 条都喊“有风险”，一次“没风险”都没输出过。误判的全是“官方腔调但零负面内容”的东西——NVIDIA 的游戏上线公告、美联储悼念格林斯潘的讣告、三年期支付研究报告。根因是训练语料做空特化，模型学到的是“官方财经文体 = 风险”，它没有“官方但不是坏事”这个概念。

怎么修好的：不是调参，是换架构。v2 又失败一次之后，模型卡里留了一句教训：“证据状态是一道结构化闸门，不是一个文本类别。”于是 v4 拆成三层——先用确定性的证据门判断（证据不足直接弃权，模型根本不被调用），再用确定性的语义策略门判断，只有前两层都拿不准的才交给小模型。把模型的活越切越小，整体准确率反而上来了。

最关键的一点：过了门我们也没让它上岗

v4 的 11 道门全过，但晋级决策仍然是 `QUALIFIED_SHADOW`（合格但保持影子态），原因写在模型卡里：`"human_labels_claimed": false`——盲测的标准答案是 AI 按规则打的，不是人打的，所以我们自己不承认它有资格出影子态。

这条弧线完整覆盖了今年新增的"AI 代码审查能力"维度（占 30%）：发现失败 → 公开失败 → 定位根因 → 换架构 → 重新盲测 → 过了门也仍然自限。四代产物、三次冻结盲测、每次的门禁读数全部在 `artifacts/` 目录里，可以现场逐个打开给老师看。

2.3 产品 KPI：证明"产品有用"而不是"工程复杂"

系统里有一套独立于工程健康指标的产品指标（`GET /api/v1/product/metrics`）：捕获延迟、重复压缩率、可引用证据覆盖率、证据闭合率、待复核队列年龄、边界违规数。

关键设计：每个指标只有两种状态——`MEASURED`（必须附样本量和数据来源）或 `UNAVAILABLE`（必须附不可用原因）。代码注释把意图写死了：

需要人工抽样才能得出的指标，就明确标为不可用，
绝不用模型输出或测试通过数凑一个替代分。

返回体里甚至有个字段直接写着 `"engineering_health_is_not_product_quality": true`（工程健康不等于产品质量）。

为什么这条值钱：它正面回答了评分细则里的"创新亮点"。大部分项目做不到这一点——因为承认"我测不出来"很难，大家更愿意随便编个数字填上去。

2.4 缺的（只能由我们现在补）

缺什么	现状	后果
三个 AI 禁飞区内核	<code>app/core/</code> 目录不存在，三个文件一行都没写	该模块分数清零，代码走查（15%）无从谈起
角色矩阵	<code>config/course_evidence_manifest.json</code> 全是 <code>null</code>	机器审计 11 项课程检查全 <code>false</code>
细粒度 Git 提交	680 个文件、6.6 万行代码，只有 15 次提交	属规则明令禁止的"大批次一次性提交"，过程规范判不合格
每人三轮计时 Bug + 一次即兴修改	0 次	占评分 15%（8% + 7%），全是现场个人项

按人的 AI 使用日志与反思报告	只有一份总述	占 5% + 4%
教师批准	未取得	选题定级悬空
真实人工双盲标注	基础设施全写好了，真实人审 0 条	模型永远出不了影子态

3. 为什么活都做完了，还需要你们

这个问题得正面说，不然后面的分工没法执行。

3.1 因为规则不允许一个人包办

课程规则里有几条是硬性的：

- "每个项目设定 3 个核心功能为 AI 禁飞区，这些功能必须手写"，验收时导师现场抽查，要求学生逐行讲解，无法讲解视为违规、该模块分数清零。
- "角色分工不等于只做一件事，每个人都要写代码"。
- "QA 必须独立设置，不能兼任"。
- 代码走查：每人 10 分钟讲解自己写的代码，导师现场提问。
- 每人必须完成三轮计时 Bug 演练和一次即兴修改。

换句话说：现有代码量再大，也换不来你个人的分数。评分里占 30% 的"代码理解与把控"全部是现场个人项，跟项目做得多好没关系，只跟"你能不能讲清楚你写的那部分"有关。

3.2 因为一个人包办对全组都是负债

再看一遍成绩公式：

个人成绩 = 五维度加权总分 × 项目难度系数(100/95/85) × 个人贡献系数(0.8-1.2)

个人贡献系数：由组内互评 + 导师评估确定

制度目的（规则原文）：避免"划水"与"带飞"同分

如果只有一个人有产出，会连锁发生三件事：

1. 其他人拿不出五类证据 → 个人贡献系数被压到 0.8 附近；
2. 团队协作维度（10%）没有真实站会、回顾、按人反思报告 → 全组一起扣；
3. 做得最多的那个人也拿不满 1.2 —— 因为"带飞"本身就是制度要惩罚的行为。

所以把工作真实分出去，不是照顾谁，是全组的最优解。这也是为什么下面的分工里，三个禁飞区是一人一个，不是一个人写完三个再教你们讲。

3.3 好消息：缺的正好是 AI 帮不上忙的部分

三个禁飞区内核都是 80–150 行的纯函数：不连网、不连数据库、不调用任何大模型，输入输出固定。难度适中，一两天能写完，但必须真的是自己写、自己想清楚，因为验收时要脱离编辑器逐行讲。

换个角度看：这是整个项目里最适合练手、也最容易讲清楚的部分。外围那 6 万行工程代码反而不好讲——那本来就是 AI 辅助生成、由人逐行验证修正的脚手架，我们已经在文档里如实声明过，不会拿它冒充手写成

果。

4. 我们怎么分工

4.1 三个角色

角色	职责（规则原文）	在我们项目里具体干什么
PO 产品负责人	需求优先级、Backlog 管理、验收标准定义	决定每个 Sprint 先做什么；定义"什么算做完"；维护产品章程和下一阶段计划
SM Scrum Master	站会组织、进度跟踪、障碍清除	主持每天 9:15 的 15 分钟站会；盯 12 天 3 个 Sprint 的节奏；谁被卡住了负责搬开障碍
QA 质量负责人	①审核测试用例设计 ②监督 AI 禁飞区执行 ③检查 Git 提交规范 ④组织代码走查	今年权重最高的角色。四项职责的对象都是别人的产出，所以必须独立

兼任规则：PO 和 SM 可以是同一人；QA 必须独立，绝对不能兼任。2 人组的标准形态是"1 人 PO+SM，1 人 QA+开发"。

4.2 我们组的安排

PO + SM + 架构负责人 + 答辩主讲

(现有工程的主要产出者)

承担 FZ3 `finality_gate.py`。

理由：PO 的三项职责在项目里已有实物对应——产品章程、下一阶段计划、目标一致性审计都由本人产出，导师追问"为什么这样设计"时答得出来。SM 兼任是因为全局进度和外部阻塞本来就掌握在手里。

不担任 QA：QA 要审别人的产出，而现有工程主体出自本人，自己审自己在评分上等于零证据。

QA（独立，不可兼任）

_____ 建议组内最较真的一位

承担 FZ2 `evidence_gate.py`。

日常职责：审核别人的测试用例设计是否合理（防止 AI 照着代码"打表"编测试）、监督三个禁飞区是不是真的手写、检查 Git 提交粒度、组织代码走查。

提醒：这个角色今年权重最高，也最容易出彩——因为你是唯一一个"专门找别人问题"的人，导师会重点问你发现了什么。建议从第一天起就建一个缺陷清单，把每次评审发现的问题记下来。

开发

承担 FZ1 `event_fingerprint.py`。

除了禁飞区，还负责外围可逆切片的开发（即兴修改题目大多落在这里）。

如果我们是 4 人组，第四人同样要有一个可讲解的代码模块——可以从"即兴修改抽题卡"里挑一个做成正式功能，比如给事件详情增加 `evidence_age_minutes` 字段并把 API、界面、测试同时补齐。

4.3 三个禁飞区的详细规格

共同约束（三个都一样）：

- 纯函数：不连网、不连数据库、不调用任何 LLM，输入输出固定
- 每个至少 20 个聚焦测试
- 预计 80–150 有效行
- 先提交失败的测试和设计说明，再提交最小实现——这是测试驱动，也是最好的过程证据
- 由非实现者完成评审并留记录
- 负责人要能脱离编辑器逐行讲

FZ1

`app/core/event_fingerprint.py`

要解决的问题：同一件事会被多个来源反复报道，官方文件也会被修订。系统必须能判断“这条新消息，是我已经见过的重复、是对旧事件的修订、还是一个全新事件”。

- 输入：规范化后的事件字段（来源、标题、摘要、日期、主体、URL）
- 输出：一个稳定指纹 + 关系判定（`DUPLICATE` / `REVISION` / `NEW`）

要能讲清的难点：标题被改了一个词，算修订还是新事件？同一件事被两个来源分别报道，怎么合并？两个不同事件指纹撞了怎么办？

可以参考（但不能抄）：`scripts/event_ledger.py` 里的 `stable_id()`；`source_revisions` 表的 `revision_kind` 字段设计（new / edit / delete）。

FZ2

`app/core/evidence_gate.py`

要解决的问题：给定一组证据，判断这些证据到底够不够支撑一个结论。这是整个产品的核心逻辑——我们敢说“证据不足时拒绝下结论”，靠的就是这个门。

- 输入：证据列表（权威等级、证据状态、有没有精确引文、彼此是否矛盾）
- 输出：`PRIMARY_SUPPORTED` / `DISCOVERY_SUPPORTED` / `INSUFFICIENT` / `HUMAN_REVIEW` + 理由码

要能讲清的核心：为什么必须 fail-closed——任何拿不准的情况都要往“更保守”走，而不是往“通过”走。这个理由要能讲出实际后果，不能只说“更安全”。

可以参考（但不能抄）：`EvidenceAgent._status()`、`derive_evidence_context()`。

FZ3

`app/core/finality_gate.py`

要解决的问题：法律事件有阶段（申请 → 受理 → 裁定 → 更正 / 撤回）。系统必须知道当前处在哪个阶段、这个阶段最高能给到什么严重程度，以及哪些状态转换是非法的。

- 输入：法律阶段、身份是否冲突、是否已具最终性
- 输出：状态 + 最高等级上限（S / A++ / A / B / C）+ no-trading 不变量

要能讲清的核心：哪些状态转换是非法的；官方更正和撤回怎么处理（这正好对应我们最强的那个演示案例）。

可以参考：`canonical_events.status` 的四态、`provisional_grade_cap` 字段。

红线：不能拿现有代码充数

`app/models/risk_router.py`、`app/services/replay.py` 这些是 AI 辅助写的产品脚手架。

`.agent/forbidden_zones.md` 里已经白纸黑字声明过：不得把它们倒签为学生手写的禁飞区实现。

规则规定：走查时若发现禁飞区代码是 AI 生成的，必须删掉手写重做，该模块分数清零。这不得冒险——三个内核加起来也就三四百行，老老实实写反而更快。

5. 你个人必须交出什么

这一节是全文最重要的。请每个人对照检查自己。

5.1 五类证据，一样都不能少

项目的执行包里定了一条通过门：每人五类证据至少各一项——设计、实现、测试、评审、讲解。只有文档、或者只有大量一次性提交，都不通过。

类别	你要拿出什么	对应评分项
设计	你那个禁飞区的设计说明：函数签名、输入输出样例、失败行为、为什么这么设计	设计文档完整性 5%
实现	你亲手写的代码，有你自己的首次提交和最终提交	核心功能 10% + 代码质量 5%
测试	至少 20 个聚焦测试，断言逻辑必须你自己想（AI 只能帮你把想好的用例翻译成代码）	测试用例设计 5% + 覆盖率 5%
评审	你评审别人代码的记录（写了什么问题、对方怎么改的）	团队协作 10%
讲解	能脱离编辑器逐行讲自己的代码，回答追问不卡壳	代码走查 15%

5.2 现场个人项：占 30%，谁也替不了你

这三项加起来 30%，全是当场做，做不出来就是没有

1. 代码走查（15%）——每人 10 分钟讲自己写的代码，导师现场提问。优秀档要求"逐行讲解无卡顿，能解释设计原因，回答追问流畅"。
2. Bug 注入（8%）——导师往代码里注入 3 个 Bug，30 分钟内定位并修复。优秀档是修好 2–3 个并能口头讲清思路。
3. 即兴修改（7%）——验收时现场给一个需求变更，30 分钟内实现并测试通过。

标准应答框架（背下来，三种场合通用）

复现 → 定位证据 → 根因 → 最小补丁 → 定向测试 → 全量回归 → 安全边界

最后一步"安全边界"是我们项目的加分项：改完之后主动说一句"这个改动不涉及交易路由，也没有让行情进入模型特征"，会显得你真的理解这个系统在防什么。

怎么练

项目里已经准备了 12 道安全题库（在 `.agent/student_execution_pack.md`），都覆盖核心流程但不触碰外发和交易能力，例如：

- 官方 RSS 里含裸 `&`，严格 XML 解析失败
- 同一标题修订后被错误当成新事件
- `received_at` 和 `published_at` 颠倒导致时间轴错乱
- P2 传闻在没有 PO/P1 证据时误获告警资格
- 备份文件能生成但隔离恢复失败
- 事件后行情字段误进入模型特征

练习规则：由非实现者抽题并保密，实现者先复现再修改。每次保存开始/结束时间、失败的测试、最小补丁、回归输出和口头根因。每人三轮，一轮都不能少。

5.3 AI 使用日志：每次都要记

这是今年新增的重点，占 5%（日志）+ 4%（反思报告）。格式已经定好，每次用 AI 都要记六项：

```
### 张三 - 事件指纹模块
- 协作模式：约束式氛围编程
- AI 使用场景：生成指纹归一化的测试脚手架
- Prompt 摘要：……
- AI 输出是否直接采用：部分采用
- 我的修改/调整：AI 用了简单字符串比较，我改成先做全角半角和空白归一化
- 遇到的问题：AI 不理解修订和新事件的区别，生成的用例全是重复场景
- 我理解这段代码的程度（1-5）：4
```

别敷衍这一项

评分细则的不合格档写着“记录缺失或明显敷衍”，优秀档要求“每次使用都记录，含协作模式 + 修改说明 + 理解程度”。最有价值的是“遇到的问题”那一栏——写清楚 AI 在哪里出错、你怎么发现的、怎么改的，这就是“AI 代码审查能力”最直接的证据。

反过来，如果你每条都写“AI 输出直接采用、没遇到问题、理解程度 5”，那等于自己承认没审查。

5.4 Git 提交：从现在起换习惯

规则要求每个功能点单独提交，禁止大批次一次性提交，单次超过 300 行变更要写明理由。导师每日检查 Git 日志。

```
feat(core): 实现事件指纹的规范化与稳定哈希
test(core): 补充修订与重复的边界用例
fix(core): 修复全角标点导致同一事件生成两个指纹
refactor(core): 把关系判定从主函数抽出为独立纯函数
docs(core): 补充禁飞区 FZ1 的设计说明与失败行为
```

格式是 `type(scope): 描述`。常用 type : `feat` 新功能、`fix` 修复、`test` 测试、`refactor` 重构、`docs` 文档。

我们现在的历史是 680 个文件只有 15 次提交，属于规则明令禁止的形态。这个改不了了（不重写历史），但从 Day 1 起的每一次提交都会被看到，所以从现在开始就得规范。

6.12 天怎么排

Day 1 方法论日 / 选题终审

- 全组到场，确定实名角色矩阵（谁 PO+SM、谁 QA、谁开发）
- 三个禁飞区定责任人 + 复核人（同一个禁飞区的责任人和复核人不能是同一人）
- 提交教师审批单，当面说明选题来源和难度定级，保存教师原始回复或签字扫描件
- 冻结三个禁飞区的函数签名、输入输出样例和失败行为——签名一旦定下就不要来回改
- 跑一次基线并存档：`pytest -q`、`collect_product_acceptance.py`
- 每个人本地把项目跑起来（见第 8 节），确认能打开页面

Day 2–3 方向实战

- 通读《组内说明书》，重点是第 1 章（一条新闻的一生）和第 3 章（AI 用在哪）
- 把自己那个禁飞区的设计想清楚，画出输入输出和状态转换
- 开始记 AI 使用日志（从今天起，不是从写代码起）

Sprint 1 (Day 4–6) 把禁飞区做出来

- 先提交失败的测试 + 设计说明，再提交最小实现
- 每个禁飞区至少 20 个聚焦测试
- 由非实现者完成评审，评审记录要落在文字上（谁评的、发现什么、怎么改的）
- Day 6 有代码走查抽查和 Sprint 回顾，AI 使用日志会被检查

Sprint 2 (Day 7–9) 练现场

- 每人完成 3 轮计时 Bug 演练，题目由非实现者从安全题库抽取并保密
- 每轮记录：开始/结束时间、失败测试、最小补丁、回归输出、口头根因
- Day 8 有第一次正式 Bug 注入测试
- Sprint 末每人提交一份 AI 使用反思报告

Sprint 3 (Day 10–11) 收口

- 每人完成 1 次即兴修改（从抽题卡里挑，或用导师现场题）
- 对当前版本重跑浏览器验收矩阵（大屏 / 桌面 / 移动 + 五页可访问性）
- 补齐 `config/course_evidence_manifest.json` 的真实路径和真实提交号
- 在 Python 3.12 环境重跑全量回归

□ 跑 `audit_course_readiness.py --require-ready` 直到返回 0

Day 12 答辩日

- 3 分钟主线演练（见第 7 节）
- 离线终端在答辩设备上真实跑一次启动 / 回放 / 停止
- 每人准备好脱稿逐行讲自己那 100 行禁飞区代码

两条纪律

考勤：每日 9:00 和 14:00 两次签到，缺勤 ≥ 3 次直接实训不合格，这一项单独考核，跟项目做得多好不好无关。

站会：每天 9:15，15 分钟，由 SM 主持。规则要求"每人带证据展示"——不是口头说"我在写指纹模块"，而是打开提交记录或测试结果。敷衍或频繁缺席会直接影响团队协作那 10%。

7. 答辩时你会被问什么

7.1 三分钟主线（PO 主讲，但每个人都要知道讲了什么）

时间	内容
0:00-0:25	问题：新闻快，但不可复核。我们做证据终端，不做交易信号。
0:25-0:55	态势总览：13 个官方源持续采集、实时健康度、13,357 条事件、no-trading 边界。
0:55-1:35	事件工作台：消息、声明、精确引文、修订链与只读行情上下文同屏。
1:35-2:05	Replay：固定 SEC 案例——证据不足时拒绝；官方更正撤回时，已经给出的告警资格会被收回。
2:05-2:35	模型治理：v1 误报 95% → 定位根因 → 换架构 → v4 降到 6.7% 且 11 道门全过 → 但标签是 AI 打的，所以仍锁在影子态。
2:35-2:50	工程可信：产品 KPI 与工程健康分开报告，测不出来就显式返回"不可用"。
2:50-3:00	结论："我们没有让模型更自信，而是让系统更会证明、更会拒绝、更能回放。"

7.2 会被问到的具体问题

Q：AI 在你们项目里到底干了什么？

两处，都不能决定结论。一个是风险路由分类器（盲测过门了，但因为标签是 AI 打的，我们仍不承认它有资格出影子态）；一个是只写摘要的本地小模型（输出要过 8 道校验，判定全归确定性规则）。证据门、抽取规则、评级、告警资格全是人和规则。AI 参与的是排序和摘要，不是结论。

Q：你们怎么保证不碰交易？

三层。数据库 CHECK 约束（插不进去）、CI 自动扫描交易写路由（提交不进去）、故障注入演练枚举 API 路由断言不存在（跑起来也没有）。

Q：这个模块是你写的吗？讲一下。

这是最关键的一问。只有你自己能准备。建议按这个顺序讲：这个函数解决什么问题 → 输入输出是什么 → 为什么选这个数据结构 → 最难的一个边界情况是什么 → 我写了哪些测试覆盖它 → 如果输入不合法会发生什么。

Q：为什么这里要 fail-closed / 为什么不直接放行？

FZ2 的负责人一定会被问到。要能讲出实际后果，比如："如果证据不足时默认放行，一条 P2 传闻就可能获得告警资格，用户会以为它是官方确认过的事实。我们宁可漏报也不误报，因为这个产品的价值就是可核验。"

Q：模型误报 95%，你们还敢展示？

那是 v1 的数字，已经修好了。当前 v4 误报 6.7%、11 道门全过。而且我们把 v1 的 40 条逐条错例、v2 的教训、v4 的架构改动全部留档了。发现失败 → 公开失败 → 定位根因 → 换架构 → 重新盲测，整条链可以现场打开看。

Q：断网了怎么演示？

4 个冻结回放案例 + 可执行离线终端，完全不依赖网络，也不依赖当天有没有真实重大事件发生。

8. 怎么把项目跑起来

```
# 要求 Python 3.12
python -m venv .venv
.venv/bin/pip install --require-hashes -r requirements-dev.lock

# 三个进程分开跑（开三个终端）
python -m uvicorn app.api.main:app --host 127.0.0.1 --port 8000
python -m streamlit run app/web/Home.py --server.port 8501
python -m app.workers.continuous --interval 300

# 只手动跑一轮采集（不会发任何消息）
python scripts/run_live_cycle.py --db data/finance_radar.sqlite3

# 只做本地健康检查，完全不联网
python -m app.workers.continuous --once --health-only

# 测试
python -m pytest -q
python -m compileall -q app scripts tests
```

入口：Web <http://127.0.0.1:8501> API 文档 <http://127.0.0.1:8000/docs>

两个千万别碰的开关

`--send`：Telegram 真实外发的开关。不加就是只算不发。演示和调试的时候不要手贱去加。

任何 `order / position / balance` 相关的路由：不要新增，CI 会自动扫描并拦截。这是项目的安全宪法，不是建议。

9. 名词表

名词	意思
P0 / P1 / P2 / P3	来源权威等级。P0 = 官方监管机构（SEC、美联储、CFTC、FDA…）；P1 = 发行人官方（公司自己发的）；P2 = 聚合发现源（线索级）；P3 = 传闻
candidate / verified weak / rejected	事件的四种状态。只有 verified 才可能进告警队列
exact passage 精确引文	从原文里抠出来的原话，不是 AI 复述的。这是整个产品的立身之本
shadow（影子）	模型在跑、在记录，但结果不影响任何正式判断
QUALIFIED_SHADOW	盲测过门了，但因为标签不是人工打的，我们主动不让它晋级
fail closed	拿不准的时候往“更保守”走。FZ2 的核心概念

MISSED_WINDOW	该采行情的时间点错过了。如实标记，不拿现在的价格假装是当时的
dry-run	只算不发。Telegram 默认就是这个
DEGRADED	部分来源失败但整轮跑完了。不等于系统挂了
lease（租约）	数据库里一把带过期时间的锁，保证同时只有一个采集周期在跑
禁飞区	课程规则术语：必须手写、AI 只许看不许写、验收时逐行讲解的核心模块
五类证据	设计 / 实现 / 测试 / 评审 / 讲解。每人至少各一项，缺一不通过

10. 几个可能想问的

项目这么大，我是不是永远搞不懂全部？

不需要搞懂全部。评分只要求你把自己写的那部分讲透，加上知道整体在做什么（这份文件 + 说明书第 1 章就够）。6 万行里有 3.4 万行是脚本、1.8 万行是测试，真正的应用核心只有 1.4 万行，而你的禁飞区是其中 100 多行。

我基础一般，能写好禁飞区吗？

能。三个都是纯函数：给定输入返回输出，不连网、不连数据库、不调模型。真正难的不是编程技巧，是把规则想清楚——什么情况算修订、证据到什么程度才够、哪些状态转换非法。这些是可以讨论的，也应该讨论。先写测试再写实现，会让这个过程简单很多。

我可以用 AI 帮我写禁飞区吗？

不能写，可以问。规则原文是“禁飞区代码不允许使用 AI 生成，但可以参考 AI 给出的思路”。所以你可以问 AI“判断两条新闻是不是同一件事，一般有哪些做法”，但最终代码必须自己敲、自己能逐行解释。走查时被发现是 AI 生成的，该模块分数清零。

为什么要先写失败的测试？

两个原因。一是这本来就是好习惯：先想清楚“什么算对”，再写“怎么做到”。二是评分需要——先有一个失败的测试提交、再有一个让它通过的实现提交，这个提交序列本身就是“测试驱动”最硬的证据，比任何文档都有说服力。

我们的代码是 AI 帮着写的，会不会有问题？

不会，规则明确允许“带已有半成品深化”。关键是如实说：Day 1 当面向教师披露现有工程是 AI 辅助生成、由人逐行验证修正，并且我们不主张它是禁飞区手写成果。这些话在 `.agent/ai_usage.md` 里已经写下来了。主动说清楚是加分，被问出来是减分。

如果我某个环节没做，会怎么样？

直接看得见。三轮计时 Bug 少一轮、少一次即兴修改、AI 日志空着——这些都在机器审计脚本

`audit_course_readiness.py` 的检查项里，跑出来就是 `false`，而且规则明确写着“不得先填 true 再补材料”。

所以没有临时补的余地，只能按时做。

Finance Radar 组内工作说明 · 2026-08-16 · 数据核对至提交 ceb9f57

配套文件：《Finance Radar 组内说明书》（技术细节，17 页） · .agent/student_execution_pack.md（现场证据模板与安全题库） · .agent/forbidden_zones.md（禁飞区声明）